Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
2ême Année Ingénieur Informatique

Matière:

Algorithmiques et complexité

Cours n° 01

Enseignant : Benyamina Ahmed

Programme de la matière :

I- Rappels

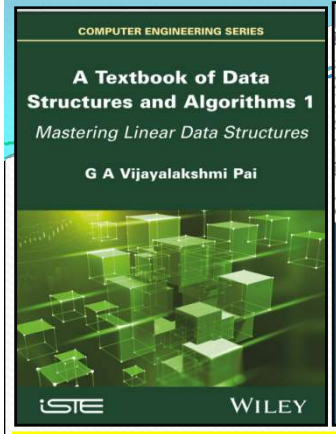
II- Notion de complexité algorithmique

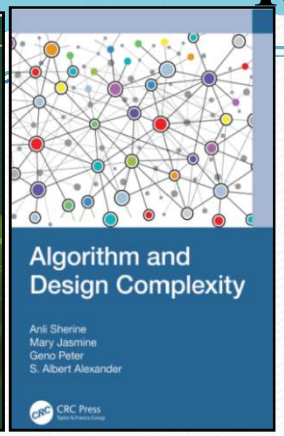
III- Algorithmes de tri

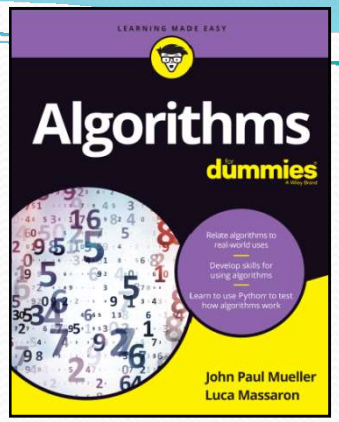
IV- Les arbres

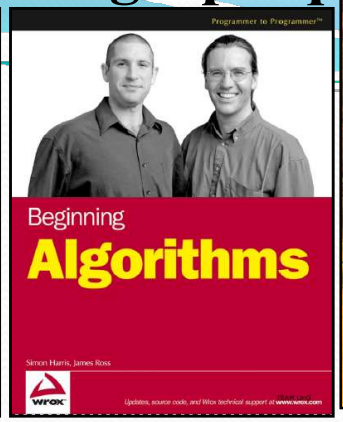
V- Les graphes

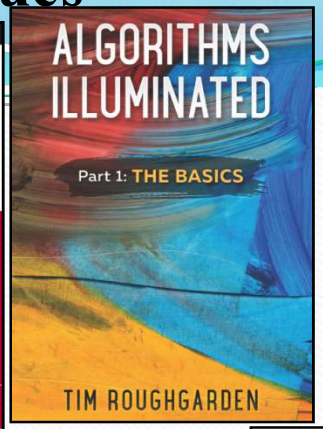
Références bibliographiques

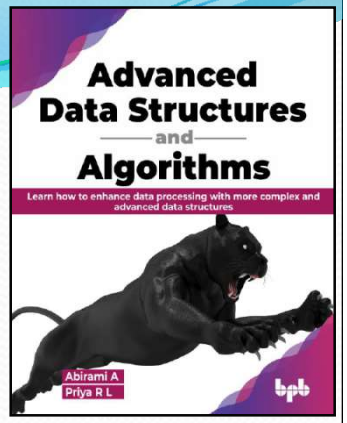


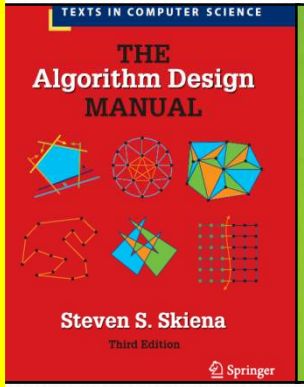


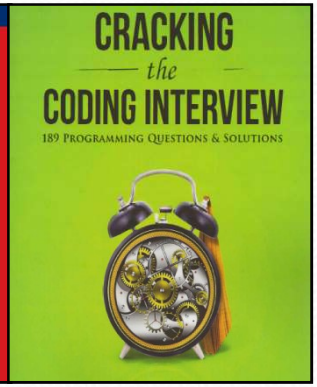


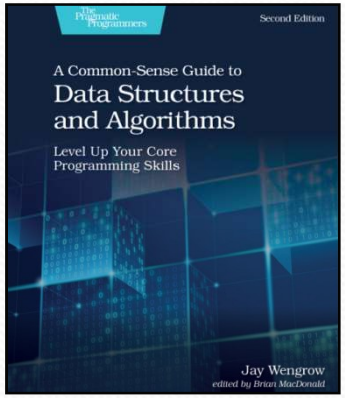


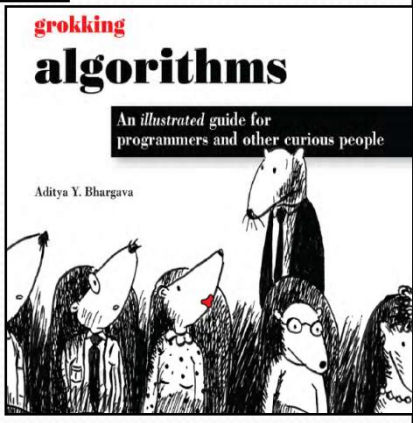












Université Tahri
Mohamed de Béchar

Polycopié réalisé par :

BENYAMINA Ahmed,
Maître de conférences.

Première partie :

Structures de données
&
Analyse d'algorithmes

Cours et exercices avec solution

- *Faculté des Sciences exactes*
- *Département de Mathématique Et Informatique*

Année Universitaire :
2019 -2020

---------- Applications (Document) ----------

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

1. **Définition d'un algorithme**
2. **Structure d'un algorithme**
3. **Les critères d'un algorithme**
4. **Etude d'un algorithme**
5. **Structures de données**

II- Notion de complexité algorithmique

III- Algorithmes de tri

IV- Les arbres

V- Les graphes

I. Rappels
1- Définition d'un algorithme

Un algorithme est une séquence finie d'instructions, dont chacune a une signification claire et peut être exécutée avec un nombre finie d’étapes dans une durée finie.

I. Rappels
2. Structure d'un algorithme

Un algorithme possède la structure suivante :

1. Etape d'entrée ;
2. Etape d'affectation ;
3. Etape de décision ;
4. Etape répétitive ;
5. Etape de sortie.

I. Rappels
3. Les critères d'un algorithme

Les critères des algorithmes sont :

1. Finitude
2. Généralité
3. Précision
4. Efficacité
5. Entrées-sorties

I. Rappels
4. Etude d'un algorithme

L'étude d'un algorithme comprend les domaines suivants:

1- La conception,

2- La validation,

3- L'analyse d'un algorithme,

4- Le test d'un programme.

I. Rappels
5 - Structures de données

<u>Définition</u> : Une structure de données est une manière d'organiser et de stocker des données en mémoire afin de les manipuler plus efficacement. Cela permet d'accéder, d'ajouter, de supprimer ou de modifier ces données de manière optimisée pour résoudre un problème spécifique.

Exemples : les listes, les tableaux, les piles, les files, les arbres et les graphes

I. Rappels
5 - Structures de données

Un type de données : Un type de données fait référence au type de valeurs que contiennent les variables d'un langage de programmation. Ainsi, les types de données de base:

entier, réel, caractère et booléen

qui sont intégrées aux langages de programmation sont appelés **types de données primitifs.**

I. Rappels
5 – Les classes des structures de données :

Les classes des structures de données :

1. Structures de données linéaires :

Exemple: Tableau, Liste, Pile et File.

2. Structures de données non linéaires:

Ces structures de données organisent les éléments de manière hiérarchique, sous forme de relations non linéaires.

Exemples : Arbre , Graphe.

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

II- Notion de complexité algorithmique

1. **Définition de la complexité algorithmique**
2. **Calculs de la complexité algorithmique**
3. **Exemples de calculs de complexités**

III- Algorithmes de tri

IV- Les arbres

V- Les graphes

II. Notion de complexité algorithmique

1- Définition de la complexité algorithmique

Un algorithme, durant son exécution, utilise deux ressources d'une machine:

- **Le temps d'exécution**
- **L'espace mémoire**

Définition: La complexité algorithmique est une mesure qui permet d'évaluer la **performance d'un algorithme** en termes de temps d'exécution (complexité temporelle) et d'utilisation de la mémoire (complexité spatiale).

II. Notion de complexité algorithmique

2- Calcul de la complexité algorithmique

Problème :

Si un algorithme utilise beaucoup de données

Solution:

il est nécessaire de choisir une « ***structure de données !!!*** » visant à minimiser ***le temps d'exécution.***

Objectif:

Dans ce qui suit, nous allons nous concentrer beaucoup plus sur *le temps d'exécution*

II. Notion de complexité algorithmique

2- Calcul de la complexité algorithmique

Facteurs affectant le temps d'exécution:

1. Machine,
2. Système d'exploitation
3. Compilateur du langage,
4. Structure de données.

Du moment qu'un algorithme peut être exécuté dans n'importe quelle machine, sous n'importe quelle système d'exploitation, compilé par n'importe quelle compilateur et ses données sont de différentes nature et taille.

➔Donc on s'intéresse seulement qu'au ***temps théorique d'exécution d'un programme*** et ***non au temps pratique***

II. Notion de complexité algorithmique

2- Calcul de la complexité algorithmique

- **Notion d'opération élémentaire** :

- Une opération élémentaire est une opération fondamentale d'un algorithme si le temps d'exécution est directement lié (par une formule mathématique) au nombre de ces opérations élémentaires. Il peut y avoir plusieurs opérations élémentaires dans un même algorithme.
- Nous pouvons comparer deux algorithmes résolvant le même problème en comparant ce nombre d'opérations élémentaires effectuées par chacun d'eux.

Complexité temporelle : notation

- Complexité = Temps mis pour transformer les données du problème considéré en un ensemble de résultats

- Complexité = {**opérations élémentaires effectuées par un algorithme donné ».**

- Inexistence de méthodologie permettant de compter les opérations élémentaires.

- Il existe seulement des ***règles*** pour évaluer la complexité temporelle.

Complexité temporelle : notation

Algorithme	Instructions	Nombre d'opération élém.NB(i_i)

sum := 0;
for i:=1 to n do
for j:=1 to n do
sum:=sum+1;
	i_1
	i_2
	i_3
	i_4
		$Nb(i_1)=n_1$
		$Nb(i_2)=n_2$
		$Nb(i_3)=n_3$
		$Nb(i_4)=n_4$

……. ……..	…… i_k	…… $Nb(i_k)=n_k$

Remarques:

- Algorithme=$i_1+ i_2+\ldots+i_k +\ldots i_n$
- Exemple d'instructions algorithmique : affectation, itération, condition,….
- Complexité temporelle : $\sum_{k=1}^{n} Nb(i_k)$

Complexité temporelle : notation

Instruction	Nombre d'opération élém.NB(i_i)
Cond= Si Exp1 alors E1 sinon E2	Nb(Cond)<Nb(Expr)+max(Nb(E1),Nb(E2))
Iter : finie bornée Nombre d'itération =Nb_Iter Instr. Dans le corps = S	Nb(Iter) = Nb(S) x *Nbr_Itération*

Exemple : Calcul du nombre d'opération de ce code écrit en langage pascal:

```
sum := 0;            (*Une opération  d'affectation = 1*)
for i:=1 to n do     (*Une opération  d'affectation  =1 *)
  for j:=1 to n do (*Une opération  d'affectation = 1 *)
    sum:=sum+a*b+1; (*Quatre opérations : Une affectation, une
                      multiplication et deux additions =4 *)
```

$1 + (2+ 4) *n*n*= (6n^2+1)$ Opérations élémentaires

On a $\mathbf{T(n)=6n^2+1}$

Définition des cas de complexité

Cas de complexité d'un problème de taille n	Complexité
Le meilleur des cas	Nb(S) x *Minorant_Nbr_Itérations*
Le pire des cas	Nb(S) x *Majorant_Nbr_Itérations*
Le cas moyen	Nb(S) x *Moyen_Nbr_Itérations*

Complexité temporelle des structures de contrôles

5- Définition des cas de complexité

Exemples:

1- Le meilleur des cas :

Les données qui font que la complexité est minimale:

- pour certains algorithmes de tri: liste déjà dans l'ordre
- résoudre une équation dont tous les coefficients sont nuls

2- Le pire des cas :

-Nombre maximum d'opération que peut faire l'algorithme

- pour certains algorithmes de tri: liste dans l'ordre inverse

3- Le cas moyen :

Le nombre moyen d'opérations calculé sur tous les problèmes de taille n est ***difficile*** à définir.

PROCHAIN COURS

Notion de complexité algorithmique